

29.yolov5模型部署实战

1. 源码分析

1.1 入口函数

相比于前面mobilenet模型的代码，代码再次封装，入口文件的代码更加简洁了。

1.1.1 图片检测

代码块

```
1  int main(int argc, char **argv)
2  {
3      // model file path
4      const char *model_file = argv[1];
5      // input img path
6      const char *img_file = argv[2];
7      // 读取图片
8      cv::Mat img = cv::imread(img_file);
9
10     // 初始化
11     Yolov5 yolo;
12     // 加载模型
13     yolo.LoadModel(model_file);
14
15     // 运行模型
16     std::vector<Detection> objects;
17     yolo.Run(img, objects);
18
19     // 显示结果
20     DrawDetections(img, objects);
21
22     // 保存结果
23     cv::imwrite("result.jpg", img);
24
25     return 0;
26 }
```

1.1.2 视频检测

```
1
2 #include <opencv2/opencv.hpp>
3
4 #include "task/yolov5.h"
5 #include "utils/logging.h"
6 #include "draw/cv_draw.h"
7
8 int main(int argc, char **argv)
9 {
10     // model file path
11     const char *model_file = argv[1];
12     // input video
13     const char *video_file = argv[2];
14     // 参数: 是否录像、绘制文字
15     const bool record = argc > 3 ? atoi(argv[3]) : false;
16
17     // 读取视频
18     cv::VideoCapture cap(video_file);
19     if (!cap.isOpened())
20     {
21         NN_LOG_ERROR("Failed to open video file: %s", video_file);
22         return -1;
23     }
24     // 获取视频尺寸、帧率
25     int width = cap.get(cv::CAP_PROP_FRAME_WIDTH);
26     int height = cap.get(cv::CAP_PROP_FRAME_HEIGHT);
27     int fps = cap.get(cv::CAP_PROP_FPS);
28     NN_LOG_INFO("Video size: %d x %d, fps: %d", width, height, fps);
29
30     // 初始化
31     Yolov5 yolo;
32     // 加载模型
33     yolo.LoadModel(model_file);
34     // 视频帧
35     cv::Mat img;
36     cv::VideoWriter writer;
37     if (record)
38     {
39         // 写入视频mp4文件
40         writer = cv::VideoWriter("result.mp4", cv::VideoWriter::fourcc('m',
41 'p', '4', 'v'), fps, cv::Size(width, height));
42     }
43
44     // all start
45     auto start_all = std::chrono::high_resolution_clock::now();
46     int frame_count = 0;
```

```

47     while (true)
48     {
49         // 开始计时
50         auto start_1 = std::chrono::high_resolution_clock::now();
51
52         // 读取视频帧
53         cap >> img;
54         if (img.empty())
55         {
56             NN_LOG_INFO("Video end.");
57             break;
58         }
59
60         // 记录读取视频帧的时间：读取视频帧的时间
61         auto end_1 = std::chrono::high_resolution_clock::now();
62         // microseconds 微秒, milliseconds 毫秒, seconds 秒, 1微妙=0.001毫秒 =
0.000001秒
63         auto elapsed_1 = std::chrono::duration_cast<std::chrono::microseconds>
(end_1 - start_1).count() / 1000.0;
64
65         // 开始计时
66         auto start_2 = std::chrono::high_resolution_clock::now();
67         // 检测结果
68         std::vector<Detection> objects;
69
70         // 运行模型
71         yolo.Run(img, objects);
72         // 绘制框，显示结果
73         DrawDetections(img, objects);
74
75         // 结束计时
76         auto end_2 = std::chrono::high_resolution_clock::now();
77         auto elapsed_2 = std::chrono::duration_cast<std::chrono::microseconds>
(end_2 - start_2).count() / 1000.0;
78
79         // 算法1：计算单张图片的总耗时
80         // 总时间
81         auto duration = std::chrono::duration_cast<std::chrono::microseconds>
(end_2 - start_1).count() / 1000.0;
82         // 计算帧率
83         auto fps = 1000.0f / duration;
84
85         // 如果计算帧率，输出帧率
86         // 输出时间：读取视频帧的时间、模型运行时间、总时间
87         NN_LOG_INFO("Method1 Time: %fms, %fms, %fms", elapsed_1, elapsed_2,
duration);
88         // 输出帧率：读取视频帧的帧率、模型运行帧率、总帧率

```

```

89         NN_LOG_INFO("Method1 FPS: %f, %f, %f", 1000.0 / elapsed_1, 1000.0 /
elapsed_2, fps);
90
91         // 算法2: 计算超过 1s 一共处理了多少张图片, 即平均帧率
92         frame_count++;
93         auto elapsed_all_2 =
std::chrono::duration_cast<std::chrono::microseconds>(end_2 -
start_all).count() / 1000.f;
94         // 每隔1秒打印一次
95         if (elapsed_all_2 > 1000)
96         {
97
98             NN_LOG_INFO("Method2 Time:%fms, FPS:%f, Frame Count:%d",
elapsed_all_2, frame_count / (elapsed_all_2 / 1000.0f), frame_count);
99             frame_count = 0;
100             start_all = std::chrono::high_resolution_clock::now();
101         }
102
103         // 如果不计算帧率, 就绘制总耗时和帧率, 写入视频帧。 (因为method2会计入这个时间)
104         if (record)
105         {
106             // 绘制总耗时和帧率
107             auto time_str = std::to_string(duration) + "ms";
108             auto fps_str = std::to_string(fps) + "fps";
109             cv::putText(img, time_str, cv::Point(50, 50),
cv::FONT_HERSHEY_PLAIN, 1.2, cv::Scalar(0xFF, 0xFF, 0xFF), 2);
110             cv::putText(img, fps_str, cv::Point(50, 100),
cv::FONT_HERSHEY_PLAIN, 1.2, cv::Scalar(0xFF, 0xFF, 0xFF), 2);
111
112             // 写入视频帧
113             writer << img;
114         }
115     }
116
117     return 0;
118 }

```

1.2 draw目录

功能：在一张图片上画出检测到的目标（比如人、车、猫等），包括一个框 + 文字标签

代码块

```

1 // 在img上画出检测结果
2 void DrawDetections(cv::Mat &img, const std::vector<Detection> &objects)
3 {

```

```

4     NN_LOG_DEBUG("draw %ld objects", objects.size());
5     for (const auto &object : objects)
6     {
7         cv::rectangle(img, object.box, object.color, 2);
8         // class name with confidence
9         std::string draw_string = object.className + " " +
std::to_string(object.confidence);
10        cv::putText(img, draw_string, cv::Point(object.box.x, object.box.y -
5), cv::FONT_HERSHEY_SIMPLEX, 0.6,
11                object.color, 2);
12    }
13 }

```

1.3 process目录

1.3.1 preprocess.cpp

代码块

```

1  /**
2   * @brief 图像预处理函数
3   * @param img 输入图像 (BGR格式)
4   * @param img_resized 输出图像 (RGB格式, 已调整大小)
5   * @param width 目标宽度
6   * @param height 目标高度
7   */
8  void imgPreprocess(const cv::Mat &img, cv::Mat &img_resized, uint32_t width,
uint32_t height)
9  {
10     // 检查输入图像是否为3通道 (BGR格式)
11     if (img.channels() != 3)
12     {
13         NN_LOG_ERROR("img has to be 3 channels"); // 记录错误日志
14         exit(-1); // 程序退出
15     }
16
17     cv::Mat img_rgb;
18     cv::cvtColor(img, img_rgb, cv::COLOR_BGR2RGB); // BGR转RGB (模型通常使用RGB
格式)
19     // 调整图像大小为模型输入尺寸
20     cv::resize(img_rgb, img_resized, cv::Size(width, height), 0, 0,
cv::INTER_LINEAR);
21 }
22
23 /**
24  * @brief YOLO的预处理函数

```

```

25  * @param img 输入图像 (BGR格式)
26  * @param width 目标宽度
27  * @param height 目标高度
28  * @param tensor 输出张量数据
29  */
30  void cvimg2tensor(const cv::Mat &img, uint32_t width, uint32_t height,
    tensor_data_s &tensor)
31  {
32      // 检查输入图像是否为3通道 (BGR格式)
33      if (img.channels() != 3)
34      {
35          NN_LOG_ERROR("img has to be 3 channels"); // 记录错误日志
36          exit(-1); // 程序退出
37      }
38
39      // BGR to RGB
40      cv::Mat img_rgb;
41      cv::cvtColor(img, img_rgb, cv::COLOR_BGR2RGB); // BGR转RGB
42
43      // resize img
44      cv::Mat img_resized;
45      NN_LOG_DEBUG("img size: %d, %d", img.cols, img.rows); // 记录调试信息
46      NN_LOG_DEBUG("resize to: %d, %d", width, height); // 记录调试信息
47      cv::resize(img_rgb, img_resized, cv::Size(width, height), 0, 0,
    cv::INTER_LINEAR); // 调整大小
48
49      // 将调整大小后的图像数据复制到张量中
50      memcpy(tensor.data, img_resized.data, tensor.attr.size);
51  }

```

1.3.2 postprocess.h

代码块

```

1  #ifndef _RKNN_ZERO_COPY_DEMO_POSTPROCESS_H_ // 防止头文件重复包含
2  #define _RKNN_ZERO_COPY_DEMO_POSTPROCESS_H_
3
4  #include <stdint.h> // 引入标准整数类型
5  #include <vector> // 引入vector容器
6
7  // 常量定义
8  #define OBJ_NAME_MAX_SIZE 16 // 物体名称最大长度
9  #define OBJ_NUMB_MAX_SIZE 64 // 最大检测物体数量
10 #define OBJ_CLASS_NUM 80 // 物体类别数量 (COCO数据集80类)
11 #define NMS_THRESH 0.45 // 非极大值抑制阈值
12 #define BOX_THRESH 0.25 // 边界框置信度阈值

```

```

13 #define PROP_BOX_SIZE (5 + OBJ_CLASS_NUM) // 每个边界框的属性大小
14
15 namespace yolov5 // YOLOv5命名空间
16 {
17     /**
18      * @brief 边界框结构体
19      */
20     typedef struct _BOX_RECT
21     {
22         int left; // 左上角x坐标
23         int right; // 右下角x坐标
24         int top; // 左上角y坐标
25         int bottom; // 右下角y坐标
26     } BOX_RECT;
27
28     /**
29      * @brief 检测结果结构体
30      */
31     typedef struct __detect_result_t
32     {
33         char name[OBJ_NAME_MAX_SIZE]; // 物体名称
34         BOX_RECT box; // 边界框
35         int id; // 物体ID
36         float prop; // 置信度
37     } detect_result_t;
38
39     /**
40      * @brief 检测结果组结构体
41      */
42     typedef struct _detect_result_group_t
43     {
44         int id; // 结果组ID
45         int count; // 检测到的物体数量
46         detect_result_t results[OBJ_NUMB_MAX_SIZE]; // 检测结果数组
47     } detect_result_group_t;
48
49     /**
50      * @brief 后处理函数
51      * @param input0 第一个输出特征图
52      * @param input1 第二个输出特征图
53      * @param input2 第三个输出特征图
54      * @param model_in_h 模型输入高度
55      * @param model_in_w 模型输入宽度
56      * @param conf_threshold 置信度阈值
57      * @param nms_threshold 非极大值抑制阈值
58      * @param scale_w 宽度缩放比例
59      * @param scale_h 高度缩放比例

```

```

60      * @param qnt_zps 量化零点
61      * @param qnt_scales 量化缩放因子
62      * @param group 检测结果组
63      * @return 处理结果 (0表示成功)
64      */
65      int post_process(int8_t *input0, int8_t *input1, int8_t *input2, int
model_in_h, int model_in_w,
66                      float conf_threshold, float nms_threshold, float scale_w,
float scale_h,
67                      std::vector<int32_t> &qnt_zps, std::vector<float>
&qnt_scales,
68                      detect_result_group_t *group);
69
70      /**
71      * @brief 后处理反初始化函数
72      */
73      void deinitPostProcess();
74  }
75  #endif // _RKNN_ZERO_COPY_DEMO_POSTPROCESS_H_

```

1.3.3 postprocess.cpp

作用：解析单个输出头（如 stride=8）的所有预测

代码块

```

1  static int process(
2      int8_t *input,          // 量化后的模型输出
3      int *anchor,            // 对应的锚框
4      int grid_h, int grid_w, // 网格尺寸
5      int height, int width,  // 模型输入尺寸
6      int stride,             // 步长 (8/16/32)
7      std::vector<float> &boxes,    // 输出: [x,y,w,h,...]
8      std::vector<float> &objProbs, // 输出: 置信度
9      std::vector<int> &classId,    // 输出: 类别ID
10     float threshold,           // 置信度阈值
11     int32_t zp, float scale    // 量化参数
12 ) {
13     int validCount = 0;
14     int grid_len = grid_h * grid_w;
15
16     // 将阈值转换为量化值 (加速比较)
17     float thres = unsigmoid(threshold);
18     int8_t thres_i8 = qnt_f32_to_affine(thres, zp, scale);
19
20     // 遍历 3 个 anchor
21     for (int a = 0; a < 3; a++) {

```



```

22         // 遍历每个网格
23         for (int i = 0; i < grid_h; i++) {
24             for (int j = 0; j < grid_w; j++) {
25                 // 获取 objectness 置信度 (第4个通道)
26                 int8_t box_confidence = input[(PROP_BOX_SIZE * a + 4) *
grid_len + i * grid_w + j];
27
28                 // 过滤低置信度
29                 if (box_confidence >= thres_i8) {
30                     int offset = (PROP_BOX_SIZE * a) * grid_len + i * grid_w +
j;
31                     int8_t *in_ptr = input + offset;
32
33                     // 解码边界框中心点 (tx, ty)
34                     float box_x = sigmoid(deqnt_affine_to_f32(*in_ptr, zp,
scale)) * 2.0 - 0.5;
35                     float box_y =
sigmoid(deqnt_affine_to_f32(in_ptr[grid_len], zp, scale)) * 2.0 - 0.5;
36
37                     // 解码宽高 (tw, th)
38                     float box_w = sigmoid(deqnt_affine_to_f32(in_ptr[2 *
grid_len], zp, scale)) * 2.0;
39                     float box_h = sigmoid(deqnt_affine_to_f32(in_ptr[3 *
grid_len], zp, scale)) * 2.0;
40
41                     // 映射到输入图像坐标系
42                     box_x = (box_x + j) * (float)stride;
43                     box_y = (box_y + i) * (float)stride;
44
45                     // 应用锚框并平方 (YOLOv5 特有)
46                     box_w = box_w * box_w * (float)anchor[a * 2];
47                     box_h = box_h * box_h * (float)anchor[a * 2 + 1];
48
49                     // 转换为 中心点+宽高 → 左上角+右下角
50                     box_x -= (box_w / 2.0);
51                     box_y -= (box_h / 2.0);
52
53                     // 寻找最高类别概率
54                     int8_t maxClassProbs = in_ptr[5 * grid_len];
55                     int maxClassId = 0;
56                     for (int k = 1; k < OBJ_CLASS_NUM; ++k) {
57                         int8_t prob = in_ptr[(5 + k) * grid_len];
58                         if (prob > maxClassProbs) {
59                             maxClassId = k;
60                             maxClassProbs = prob;
61                         }
62                     }

```

```

63
64         // 再次过滤 (类别置信度)
65         if (maxClassProbs > thres_i8) {
66             // 总置信度 = objectness * class_prob
67             objProbs.push_back(
68                 sigmoid(deqnt_affine_to_f32(maxClassProbs, zp,
scale)) *
69                 sigmoid(deqnt_affine_to_f32(box_confidence, zp,
scale))
70             );
71             classId.push_back(maxClassId);
72             validCount++;
73
74             // 保存边界框 [x, y, w, h]
75             boxes.push_back(box_x);
76             boxes.push_back(box_y);
77             boxes.push_back(box_w);
78             boxes.push_back(box_h);
79         }
80     }
81 }
82 }
83 }
84 return validCount;
85 }

```

整合三个尺度的输出，并执行 NMS

代码块

```

1  int post_process(
2      int8_t *input0, int8_t *input1, int8_t *input2, // 三个输出头
3      int model_in_h, int model_in_w,                // 模型输入尺寸
4      float conf_threshold, float nms_threshold,      // 阈值
5      float scale_w, float scale_h,                  // 原图缩放比例
6      std::vector<int32_t> &qnt_zps,                  // 量化零点
7      std::vector<float> &qnt_scales,                 // 量化尺度
8      detect_result_group_t *group                   // 输出结果
9  ) {
10     // 初始化标签 (只执行一次)
11     static int init = -1;
12     if (init == -1) {
13         // loadLabelName(LABEL_NALE_TXT_PATH, labels); // 被注释, 用内置标签
14         init = 0;
15     }
16

```

```

17     memset(group, 0, sizeof(detect_result_group_t));
18
19     // 存储所有检测结果
20     std::vector<float> filterBoxes; // [x,y,w,h, x,y,w,h, ...]
21     std::vector<float> objProbs;    // 置信度列表
22     std::vector<int> classId;       // 类别ID列表
23
24     // 处理 stride=8
25     int stride0 = 8;
26     int grid_h0 = model_in_h / stride0;
27     int grid_w0 = model_in_w / stride0;
28     int validCount0 = process(input0, (int *)anchor0, grid_h0, grid_w0,
29                               model_in_h, model_in_w, stride0,
30                               filterBoxes, objProbs, classId,
31                               conf_threshold, qnt_zps[0], qnt_scales[0]);
32
33     // 处理 stride=16
34     int stride1 = 16;
35     int grid_h1 = model_in_h / stride1;
36     int grid_w1 = model_in_w / stride1;
37     int validCount1 = process(input1, (int *)anchor1, grid_h1, grid_w1,
38                               model_in_h, model_in_w, stride1,
39                               filterBoxes, objProbs, classId,
40                               conf_threshold, qnt_zps[1], qnt_scales[1]);
41
42     // 处理 stride=32
43     int stride2 = 32;
44     int grid_h2 = model_in_h / stride2;
45     int grid_w2 = model_in_w / stride2;
46     int validCount2 = process(input2, (int *)anchor2, grid_h2, grid_w2,
47                               model_in_h, model_in_w, stride2,
48                               filterBoxes, objProbs, classId,
49                               conf_threshold, qnt_zps[2], qnt_scales[2]);
50
51     int validCount = validCount0 + validCount1 + validCount2;
52     if (validCount <= 0) return 0;
53
54     // 创建索引数组 [0, 1, 2, ..., validCount-1]
55     std::vector<int> indexArray;
56     for (int i = 0; i < validCount; ++i) {
57         indexArray.push_back(i);
58     }
59
60     // 按置信度降序排序
61     quick_sort_indice_inverse(objProbs, 0, validCount - 1, indexArray);
62
63     // 对每个类别分别执行 NMS

```

```

64     std::set<int> class_set(std::begin(classId), std::end(classId));
65     for (auto c : class_set) {
66         nms(validCount, filterBoxes, classId, indexArray, c, nms_threshold);
67     }
68
69     // 填充最终结果结构
70     int last_count = 0;
71     group->count = 0;
72     for (int i = 0; i < validCount; ++i) {
73         if (indexArray[i] == -1 || last_count >= OBJ_NUMB_MAX_SIZE) continue;
74
75         int n = indexArray[i]; // 实际检测框索引
76
77         // 转换为左上-右下坐标
78         float x1 = filterBoxes[n * 4 + 0];
79         float y1 = filterBoxes[n * 4 + 1];
80         float x2 = x1 + filterBoxes[n * 4 + 2];
81         float y2 = y1 + filterBoxes[n * 4 + 3];
82         int id = classId[n];
83         float obj_conf = objProbs[i]; // 注意: 这里用排序后的 i
84
85         // 缩放回原图尺寸
86         group->results[last_count].box.left = (int)(clamp(x1, 0, model_in_w)
/ scale_w);
87         group->results[last_count].box.top = (int)(clamp(y1, 0, model_in_h)
/ scale_h);
88         group->results[last_count].box.right = (int)(clamp(x2, 0, model_in_w)
/ scale_w);
89         group->results[last_count].box.bottom = (int)(clamp(y2, 0, model_in_h)
/ scale_h);
90         group->results[last_count].prop = obj_conf;
91
92         // 复制类别名称
93         const char *label = labels[id];
94         strncpy(group->results[last_count].name, label, OBJ_NAME_MAX_SIZE);
95
96         last_count++;
97     }
98     group->count = last_count;
99
100     return 0;
101 }

```

1.4 task目录

1.4.1 yolov5.h

代码块

```
1
2  #ifndef RK3588_DEMO_YOLOV5_H
3  #define RK3588_DEMO_YOLOV5_H
4
5  #include "types/yolo_datatype.h"
6  #include "engine/engine.h"
7
8  class YOLOv5
9  {
10 public:
11     YOLOv5();
12     ~YOLOv5();
13
14     nn_error_e LoadModel(const char *model_path);           // 加载
15     // 模型
16     nn_error_e Run(const cv::Mat &img, std::vector<Detection> &objects); // 运行
17     // 模型
18 private:
19     nn_error_e Preprocess(const cv::Mat &img);
20     // 图像预处理
21     nn_error_e Inference();
22     // 推理
23     nn_error_e Postprocess(const cv::Mat &img, std::vector<Detection>
24     &objects); // 后处理
25
26     // 输入张量：存储预处理后的图像数据
27     tensor_data_s input_tensor_;
28     // 输出张量列表：YOLOv5 通常有 3 个输出 (对应 stride=8/16/32)
29     std::vector<tensor_data_s> output_tensors_;
30     // 量化参数：用于反量化 (int8 → float)
31     // out_zps_[i] 和 out_scales_[i] 对应第 i 个输出张量
32     std::vector<int32_t> out_zps_; // Zero Point (零点偏移)
33     std::vector<float> out_scales_; // Scale (缩放因子)
34     // 推理引擎智能指针 (如 RKNN Engine)
35     std::shared_ptr<NNEngine> engine_;
36 };
37
38 #endif // RK3588_DEMO_YOLOV5_H
39
```

1.4.2 yolov5.cpp

功能：从模型加载 → 图像预处理 → NPU 推理 → 后处理 → 输出检测结果 的全流程。

主要分为以下 6 个功能模块：

1. 辅助转换函数：DetectionGrp2DetectionArray
2. 构造/析构函数：资源初始化与释放
3. 模型加载（LoadModel）
4. 预处理（Preprocess）
5. 推理（Inference）
6. 后处理（Postprocess） + 主入口（Run）

代码块

```
1
2  #include "yolov5.h"
3
4  #include <memory>
5
6  #include "utils/logging.h"
7  #include "process/preprocess.h"
8  #include "process/postprocess.h"
9
10 #include <ctime>
11
12 void DetectionGrp2DetectionArray(yolov5::detect_result_group_t &det_grp,
13 std::vector<Detection> &objects)
14 {
15     // 根据当前系统时间生成随机数种子
16     std::srand(static_cast<unsigned int>(std::time(nullptr)));
17
18     // 遍历每个检测结果
19     for (int i = 0; i < det_grp.count; i++)
20     {
21         Detection det;
22         det.className = det_grp.results[i].name;
23
24         // 计算边界框的宽度和高度
25         det.box = cv::Rect(det_grp.results[i].box.left,
26                             det_grp.results[i].box.top,
27                             det_grp.results[i].box.right -
28                             det_grp.results[i].box.left,
29                             det_grp.results[i].box.bottom -
30                             det_grp.results[i].box.top);
31
32         det.confidence = det_grp.results[i].prop;
33         det.class_id = 0;
```

```

31         // 生成随机颜色
32         det.color = cv::Scalar(rand() % 255, rand() % 255, rand() % 255);
33         objects.push_back(det);
34     }
35 }
36
37 // 构造函数
38 YOLOV5::YOLOV5()
39 {
40     engine_ = CreateRKNNEngine();
41     input_tensor_.data = nullptr;
42 }
43 // 析构函数
44 YOLOV5::~YOLOV5()
45 {
46     if (input_tensor_.data != nullptr)
47     {
48         free(input_tensor_.data);
49         input_tensor_.data = nullptr;
50     }
51     for (auto &tensor : output_tensors_)
52     {
53         free(tensor.data);
54         tensor.data = nullptr;
55     }
56 }
57
58 // 加载模型，获取输入输出属性
59 nn_error_e YOLOV5::LoadModel(const char *model_path)
60 {
61     // 调用引擎加载模型文件
62     auto ret = engine_>LoadModelFile(model_path);
63     if (ret != NN_SUCCESS)
64     {
65         NN_LOG_ERROR("yolo load model file failed");
66         return ret;
67     }
68     // 获取输入张量的属性信息
69     auto input_shapes = engine_>GetInputShapes();
70
71     // 检查输入张量数量是否正确
72     if (input_shapes.size() != 1)
73     {
74         NN_LOG_ERROR("yolo input tensor number is not 1, but %ld",
input_shapes.size());
75         return NN_RKNN_INPUT_ATTR_ERROR;
76     }

```

```

77
78 // 初始化输入张量并分配内存
79 nn_tensor_attr_to_cvmg_input_data(input_shapes[0], input_tensor_);
80 input_tensor_.data = malloc(input_tensor_.attr.size);
81
82 // 获取输出张量的属性信息
83 auto output_shapes = engine_>GetOutputShapes();
84
85 // 遍历每个输出张量
86 for (int i = 0; i < output_shapes.size(); i++)
87 {
88     //
89     tensor_data_s tensor;
90     tensor.attr.n_elems = output_shapes[i].n_elems;
91     tensor.attr.n_dims = output_shapes[i].n_dims;
92     for (int j = 0; j < output_shapes[i].n_dims; j++)
93     {
94         tensor.attr.dims[j] = output_shapes[i].dims[j];
95     }
96     // 检查张量类型是否为 int8 (量化后的数据类型)
97     if (output_shapes[i].type != NN_TENSOR_INT8)
98     {
99         NN_LOG_ERROR("yolo output tensor type is not int8, but %d",
output_shapes[i].type);
100         return NN_RKNN_OUTPUT_ATTR_ERROR;
101     }
102
103     // 分配张量内存
104     tensor.attr.type = output_shapes[i].type;
105     tensor.attr.index = i;
106     tensor.attr.size = output_shapes[i].n_elems *
nn_tensor_type_to_size(tensor.attr.type);
107     tensor.data = malloc(tensor.attr.size);
108     // 保存量化参数 (用于后处理反量化)
109     output_tensors_.push_back(tensor);
110     out_zps_.push_back(output_shapes[i].zp);
111     out_scales_.push_back(output_shapes[i].scale);
112 }
113 return NN_SUCCESS;
114 }
115
116 // 图像预处理
117 nn_error_e YoloV5::Preprocess(const cv::Mat &image)
118 {
119     // 将预处理后的结果放入 input_tensor_ 中
120     cvimg2tensor(image, input_tensor_.attr.dims[2],
input_tensor_.attr.dims[1], input_tensor_);

```



```

121     return NN_SUCCESS;
122 }
123
124 // 推理
125 nn_error_e Yolov5::Inference()
126 {
127     std::vector<tensor_data_s> inputs;
128     // 将input_tensor_放入inputs中
129     inputs.push_back(input_tensor_);
130     // 运行模型
131     engine_>Run(inputs, output_tensors_, false);
132     return NN_SUCCESS;
133 }
134
135 // 运行模型
136 nn_error_e Yolov5::Run(const cv::Mat &img, std::vector<Detection> &objects)
137 {
138     Preprocess(img);           // 图像预处理
139     Inference();               // 推理
140     Postprocess(img, objects); // 后处理
141     return NN_SUCCESS;
142 }
143
144 // 后处理
145 nn_error_e Yolov5::Postprocess(const cv::Mat &img, std::vector<Detection>
&objects)
146 {
147     // 获取模型输入尺寸
148     int height = input_tensor_.attr.dims[1]; // H
149     int width = input_tensor_.attr.dims[2]; // W
150
151     // 计算缩放因子（用于将坐标映射回原图）
152     float scale_w = height * 1.f / img.cols;
153     float scale_h = width * 1.f / img.rows;
154
155     // 创建检测结果结构体
156     yolov5::detect_result_group_t detections;
157
158     yolov5::post_process((int8_t *)output_tensors_[0].data, // 解析第一个输出特征
图
159                             (int8_t *)output_tensors_[1].data, // 解析第二个输出特征
图
160                             (int8_t *)output_tensors_[2].data, // 解析第三个输出特征
图
161                             height, width, // 特征图尺寸
162                             BOX_THRESH, NMS_THRESH, // 置信度阈值和 NMS 阈值
163                             scale_w, scale_h, // 缩放因子（用于将坐标映射回原图）

```

```

164         out_zps_, out_scales_, // 量化参数 (用于后处理反量化)
165         &detections); // 存储检测结果
166
167         // 调用 DetectionGrp2DetectionArray 将结果转换为 Detection 对象数组
168         DetectionGrp2DetectionArray(detections, objects);
169         return NN_SUCCESS;
170     }
171

```

1.5 types目录

1.5.1 yolo_datatype.h

为什么需要2个结构体？

1.数据表示方式不同：

- nn_object_s：使用中心点坐标+宽高的表示方式，符合模型输出格式
- Detection：使用左上角坐标+宽高的表示方式（通过 cv::Rect），符合OpenCV绘图函数的需求

2.用途不同：

- nn_object_s：更适合模型内部处理，结构紧凑
- Detection：更适合最终的结果展示，包含了可视化所需的额外信息

代码块

```

1  typedef struct _nn_object_s {
2      float x;           // 物体中心的x坐标
3      float y;           // 物体中心的y坐标
4      float w;           // 物体的宽度
5      float h;           // 物体的高度
6      float score;       // 检测的置信度 (0-1之间的值, 表示模型对检测结果的确定程度)
7      int class_id;      // 物体的类别ID (如0表示人, 1表示车等)
8  } nn_object_s;
9
10 struct Detection
11 {
12     int class_id{0};     // 物体的类别ID, 默认值为0
13     std::string className{}; // 物体的类别名称 (如"person"、"car"等), 默认值为空
14                               // 字符串
15     float confidence{0.0}; // 检测的置信度, 默认值为0.0
16     cv::Scalar color{};    // 绘制物体时使用的颜色, 默认值为黑色
17     cv::Rect box{};        // 物体的边界框 (矩形), 包含左上角坐标、宽度和高度

```

```
17 };  
18  
19 #endif //RK3588_DEMO_NN_DATATYPE_H
```

1.6 engine目录

和上一节的一样，参考上一节内容。

1.7 utils目录

和上一节的一样，参考上一节内容。

2. 代码编译

2.1 方式一

交叉编译

代码块

```
1  #!/bin/bash  
2  set -e # 出错立即退出  
3  
4  export PATH=/opt/tool_chain/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-  
   gnu/bin:$PATH  
5  rm -rf build  
6  
7  cmake -S . -B build \  
8      -DCMAKE_C_COMPILER=aarch64-rockchip1031-linux-gnu-gcc \  
9      -DCMAKE_CXX_COMPILER=aarch64-rockchip1031-linux-gnu-g++  
10  
11  cmake --build build -j4  
12  cmake --install build  
13  
14  echo "✅ 打包完成! 部署目录: $(pwd)/build/install"  
15
```

执行./build-linux.sh就行，然后把编译出来的install直接传输到开发板上面就行。

2.2 方式二

板端编译

代码块

```
1 sudo apt install cmake build-essential
2 sudo apt install libopencv-dev
```

代码块

```
1 #!/bin/bash
2
3 rm -rf build
4 cmake -S . -B build
5
6 cmake --build build -j8
7
8 echo "✅ 打包完成! 部署目录:build"
```

3. 结果



```
[NN_DEBUG] img size: 1280, 720
[Nn_DEBUG] resize to: 640, 640
[Nn_DEBUG] rknn running...
[Nn_DEBUG] output num: 3
[Nn_DEBUG] output size: 1632000
[Nn_DEBUG] output want_float: 0
[Nn_DEBUG] output[0] size=1632000
[Nn_DEBUG] output size: 408000
[Nn_DEBUG] output want_float: 0
[Nn_DEBUG] output[1] size=408000
[Nn_DEBUG] output size: 102000
[Nn_DEBUG] output want_float: 0
[Nn_DEBUG] output[2] size=102000
[Nn_DEBUG] draw 21 objects
[Nn_INFO] Method1 Time: 2.555000ms, 41.136000ms, 43.692000ms
[Nn_INFO] Method1 FPS: 391.389432, 24.309607, 22.887485
[Nn_INFO] Method2 Time:1035.223022ms, FPS:26.081337, Frame Count:27
```